

Exercice 1 :**Arbres préfixes binaires**

On se propose de décrire une structure de données pour représenter les sous-ensembles finis de $\mathbb{N}^*$. Il s'agit (avec quelques subtilités dans lesquelles nous ne rentrerons pas) des *arbres préfixes*¹ binaires. Nous mentionnerons plus tard l'utilisation de forêts préfixes pour implémenter des tableaux associatifs.

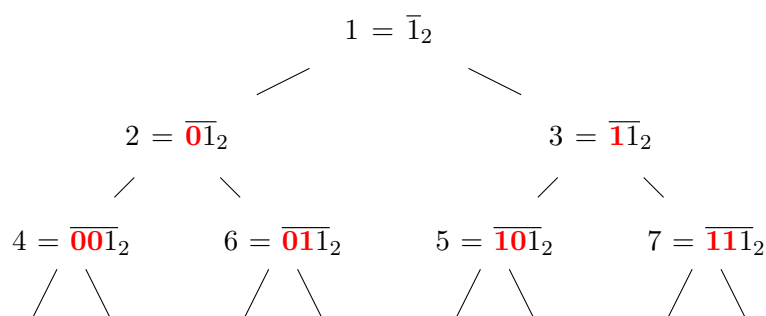
Pour cela, pour chaque entier $n \in \mathbb{N}^*$, on procède ainsi pour trouver le nœud d'un arbre binaire parfait fictif de hauteur arbitrairement grande qui lui correspond. On se donne $A = \text{Noeud}(-, A_g, A_d)$ cet arbre parfait fictif.

Si $n = 1$, la racine de A représente n . Sinon :

1. Si n est pair, alors n est représenté par le nœud de A_g qui représente $\frac{n}{2}$;
2. sinon, n est représenté par le nœud de A_d qui représente $\frac{n-1}{2}$.

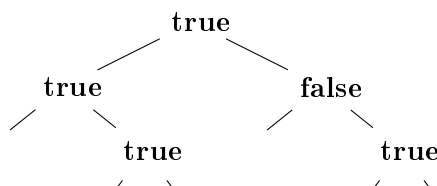
À titre d'exemple, voici où se retrouvent placés les entiers de 1 à 7. Comme on le voit, l'ordre des entiers **n'est pas le même** que l'ordre d'un parcours en largeur sur un arbre complet.

On indique leur représentation binaire petit bout, qui met en valeur le caractère d'arbre préfixe : le chemin parcouru depuis la racine jusqu'à un nœud indique un préfixe (avec 0 pour "gauche" et 1 pour "droite") de l'écriture petit bout de tous les entiers qui sont représentés par ce nœud ou ses descendants.



On choisit de représenter en OCaml une partie $F \subset \mathbb{N}^*$ par un arbre binaire étiqueté par des booléens, ou pour chaque nœud l'étiquette indique la présence ou l'absence de l'entier correspondant selon la méthode décrite ci-dessus dans F . Si un nœud n'existe pas dans l'arbre (on a **Vide** à la place), alors tous les entiers qui devraient être dans ce sous-arbre sont absents de F .

L'ensemble $\{1; 2; 6; 7\}$ sera donc représenté par l'arbre ci-dessous :



Par la suite, on identifiera un ensemble d'entiers avec l'arbre préfixe binaire qui le représente. On pose :

```

type 'a arbre_bin = Vide | Noeud of 'a arbre_bin * 'a * 'a arbre_bin
type set = bool arbre_bin

```

1. Aussi appelés *tries* (singulier *trie*), prononcé comme les mots anglais *try* et *tries*, pour ne pas confondre avec le concept de tri (c'est à dire, ordonner une séquence de valeurs).

L'un des inventeurs du concept, qui a proposé le nom "trie" d'après le mot "retrieval", proposait de le prononcer comme la syllabe en question. La plupart des personnes censées ont immédiatement fait remarquer qu'avoir un concept distinct des arbres désigné par un mot homophone de "tree" n'était pas une idée brillante.

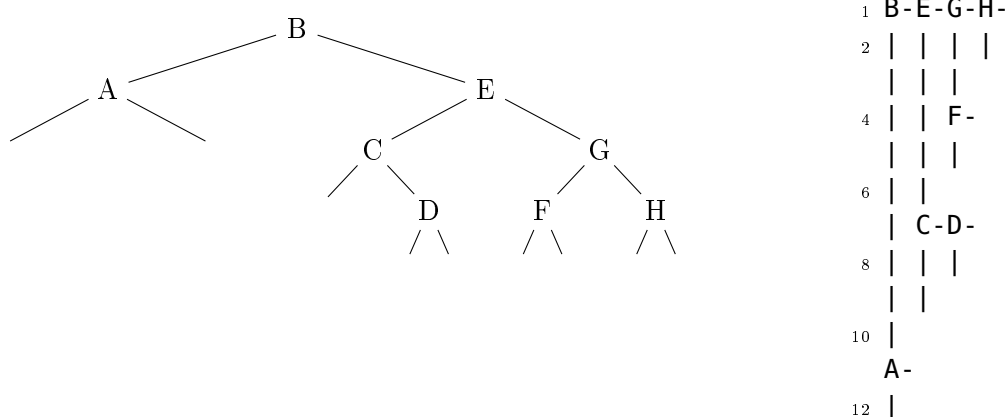
1. Représenter sous forme d'un arbre préfixe binaire les ensembles suivants. On choisira à chaque fois l'arbre binaire de plus petite taille parmi ceux qui conviennent :
 - (a) $\{42\}$;
 - (b) $\llbracket 1; 10 \rrbracket$;
 - (c) $\{8; 15\}$.
2. Écrire une fonction **mem** : **set** -> **int** -> **bool** qui prend en arguments un ensemble fini $F \subset \mathbb{N}^*$ (représenté par un arbre préfixe binaire) et un entier n , et qui renvoie un booléen déterminant l'appartenance de n à F .
3. Écrire une fonction **singleton** : **int** -> **set** qui prend en argument $n \in \mathbb{N}^*$ et renvoie l'ensemble $\{n\}$.
4. (a) Reprendre la question 1 avec la représentation sous forme d'arbre préfixe binaire de $\llbracket 1; 10 \rrbracket$. Faire un schéma de la mémoire indiquant comment on peut réutiliser certains nœuds pour obtenir un arbre préfixe binaire représentant $\llbracket 1; 10 \rrbracket \cup \{42\}$.
(b) Écrire une fonction **add** : **set** -> **int** -> **set** qui prend en arguments un ensemble fini $F \subset \mathbb{N}^*$ et $n \in \mathbb{N}^*$ et renvoie l'ensemble $F \cup \{n\}$.
5. Écrire une fonction **list_to_set** : **int list** -> **set** construisant un ensemble à partir de la liste des entiers qu'il contient.
6. Écrire une fonction **set_to_array** : **set** -> **bool array** qui représente l'ensemble fini $F \subset \mathbb{N}^*$ sous la forme d'un tableau de booléens, de taille 2^{h+1} où h est la hauteur de l'arbre (il se peut qu'aucun des entiers du dernier niveau ne soient dans F si l'étiquette est **false**, cf la question 8).
7. (a) Comme dans la question 4a, faire un schéma de la mémoire indiquant comment on peut réutiliser certains nœuds pour obtenir un arbre préfixe binaire représentant $\llbracket 1; 10 \rrbracket \cup \{42\}$ à partir de l'arbre préfixe binaire $\llbracket 1; 10 \rrbracket$.
(b) En s'inspirant de **add**, écrire une fonction **remove** : **set** -> **int** -> **set** prenant en arguments un ensemble fini $F \subset \mathbb{N}^*$ et $n \in \mathbb{N}^*$ et renvoyant l'ensemble $F \setminus \{n\}$ (si n n'appartient pas à F , cette fonction ne doit pas échouer mais renvoyer le même ensemble F).
8. (*Plus dur*) La fonction **remove** peut renvoyer un arbre avec des branches mortes (uniquement avec des nœuds à **false**), écrire une fonction **prune** : **set** -> **set** qui prend en argument un arbre préfixe binaire représentant un ensemble, et renvoie un arbre préfixe binaire représentant le même ensemble mais sans branches mortes (donc de taille minimale).
9. Écrire une fonction **union** : **set** -> **set** -> **set** réalisant l'union de deux ensembles.
10. Écrire une fonction **intersection** : **set** -> **set** -> **set** réalisant l'intersection de deux ensembles.

Exercice 2 :**Affichage d'arbres binaires**

On souhaite afficher une représentation d'un arbre binaire. On dispose pour ceci du fichier **exo2.c** qui contient déjà une implémentation des arbres binaires dont les étiquettes sont des caractères, ainsi que la définition de l'arbre utilisé pour les exemples.

1. Dans cette 1ère question, on souhaite que l'arbre soit affiché avec les enfants droits à droite de leur parent et les enfants gauche en-dessous.

Ainsi, pour l'arbre ci-dessous, on aura le résultat à droite :

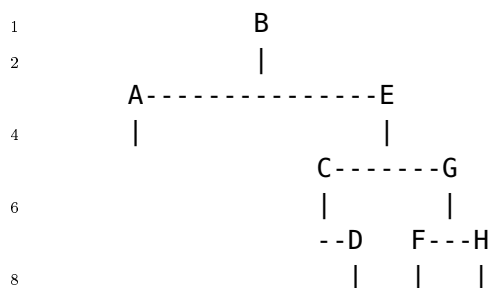


- (a) Si on procède en effectuant un parcours récursif en profondeur de l'arbre, de quelles informations supplémentaires (sur la position du nœud dans l'arbre) a-t-on besoin pour afficher un nœud dans sa ligne ? Pourquoi ?
- (b) Écrire une fonction récursive **void print_aux(arbre_bin* n, int p_droite)** qui prend en argument un nœud N (sous forme d'arbre binaire) d'un certain arbre binaire A , **p_droite** le nombre de fois où il faut aller à droite depuis la racine pour atteindre N (on ne précise pas la profondeur totale), et affiche le sous-arbre de racine N .

Cette fonctions doivent respecter le fait que les lignes où N sera affiché contiennent des liens du reste de l'arbre, par exemple le sous-arbre dont la racine est d'étiquette **E** sera affiché sur une ligne déjà commencée.

De plus, il faut penser (pour éviter, sur l'exemple, de relier C directement à D), à laisser une ligne "vide" entre un sous-arbre droit et le sous-arbre-gauche non vide qui suit.

- (c) Écrire une fonction **void print_dfs(arbre_bin* a)** qui affiche ainsi un arbre binaire.
2. On cherche maintenant à afficher l'arbre avec les nœuds de même profondeur sur la même ligne. Ainsi, avec l'exemple précédent, on voudrait afficher :



Pour cela, on va effectuer un parcours en largeur de l'arbre, en utilisant en guise de files des tableaux, de type **arbre_bin****, de longueur 2^h , où h est la hauteur de l'arbre. En effet, il y a au plus 2^p nœuds de profondeur p dans l'arbre, pour $0 \leq p \leq h$.

De plus, pour afficher correctement l'arbre, on fera comme si les sous-arbres vides à la profondeur $p < h$ avaient en réalité deux enfants vides à la profondeur $p + 1$. En d'autre termes, on considère l'arbre binaire complet le plus petit possible qui a tous les nœuds de l'arbre.

Attention : On ne fera cette opération que pour manipuler les files : évidemment on ne veut pas afficher les liens correspondants.

- (a) On décide arbitrairement que les feuilles les plus profondes sont espacées de 3 caractère, et que la feuille la plus à gauche (si elle existe) touche le bord de la fenêtre d'affichage.

Déterminer, en fonction de h et p , l'espacement entre deux nœuds consécutifs (dans l'ordre du parcours en largeur) de profondeur p , ainsi que l'espacement entre le bord de la fenêtre et le nœud de profondeur p le plus à gauche.

- (b) Comment afficher un nœud qui est un enfant gauche, un nœud qui est un enfant droit, ou un sous-arbre-vide, d'après l'exemple plus haut et les valeurs que vous venez de calculer ?

Il est conseillé de faire un schéma où les caractères affichés lors du traitement de chaque nœud dans l'ordre du parcours en largeur sont indiqués.

- (c) Écrire une fonction `void print_bfs(arbre_bin* a)` qui affiche un arbre selon ce schéma.

Attention au cas du niveau de profondeur 0, qu'il faut traiter à part (la racine ne correspondant ni au cas d'un enfant droit ni au cas d'un enfant gauche).